

ROS 2 Global Path Planning Benchmarking Suite

A Technical Deep-Dive & Architecture Reference Manual

Author: Antigravity AI Pair Partner

Target Framework: ROS 2 Humble

Status: Production Completed & Verified

Date: July 2026

This manual compiles the complete system architecture, codebase design patterns, and logical math definitions for the ROS 2 Global Path Planning Benchmarking Suite. Use this reference to understand package dependencies, launch orchestrations, costmap conversions, A* planning execution, reference path criteria, dynamic obstacle testing, and report generation.

Part 1: Build & Package Configuration

ROS 2 relies on structured configurations to manage dependencies and system integration. This harness uses three files at its package root to declare library packages, executable entrypoints, and library flags.

1. package.xml

This configuration file lists the dependencies required to build and execute the benchmarking nodes. Key nodes run on rclpy (ROS 2 Python Client Library) and communicate using standard messages and actions. The dependencies include: **rclpy**, **nav_msgs** (for costmap OccupancyGrids and planning Paths), and **nav2_msgs** (for the compute_path_to_pose Nav2 Action).

2. setup.py

Written using setuptools, this script handles node installation and maps internal Python modules to ROS 2 executable targets. It performs two critical roles:

- **Data Files Mapping:** Copies the config templates and ROS 2 launch scripts into the installation share folder so that ROS 2 can find them globally.
- **Entry Points Registration:** Binds command-line targets to functions. It registers benchmarker to benchmarker.py:main, and mock_planner to mock_planner.py:main.

Part 2: Launch Orchestration Pipeline

The launch script (benchmark.launch.py) orchestrates the setup and execution lifecycles. It acts as a synchronous supervisor. The launch sequence proceeds as follows:

1. **Declare Launch Parameters:** Exposes user-configurable CLI flags including verify, use_astar, scenario_id, and clean.
2. **Stale Node Termination:** If clean:=true, it issues a synchronous system pkill command. It scans for and forcefully terminates lingering simulator processes (Gazebo, gzserver) or Nav2 planning containers from previous crashes. This ensures a clean workspace and prevents port collisions.
3. **Config Path Resolution:** Automatically resolves relative paths to absolute locations so configuration scenarios can be successfully loaded.
4. **Execution Routing:** If verify:=true, it skips starting the planner node entirely and launches only the benchmarker in validation mode. Otherwise, it spawns both the planning server and the benchmarker nodes simultaneously.

Part 3: Map Translation & Scenario Configuration

The benchmarking system uses a configuration-driven approach where maps and start/goal positions are declared in a YAML schema and converted dynamically.

1. Configuration Schema (scenarios.yaml)

Each test case specifies a map_image, the physical coordinate resolution (meters per pixel), the origin coordinate offset, and physical parameters like robot_radius (e.g., 0.3m for Jackal/rover physical sizing). It also details the start and goal positions, the perfect reference_path coordinates, and optional dynamic obstacle insertion triggers.

2. Converting 2D Grayscale PNGs to ROS OccupancyGrids

To translate static image layouts into costmaps for planning nodes, the benchmarker processes the pixels:

- Black pixels (value 0) are converted to lethal cost **100**.
- White pixels (value 255) are converted to free space **0**.
- Gray values (1-254) are scaled linearly to intermediate costs (1-99).
- The matrix is flipped vertically using `np.flipud` because image indices start top-left, whereas ROS frames start bottom-left.

3. Reference Path (Ground Truth) Definition

To evaluate if the planner's length is optimal, we compare it against a ground-truth `reference_path`. This path is defined via three primary methodologies:

- **Offline Global Optimization:** Using computationally heavy planners (like Dijkstra or CHOMP) offline with infinite time budgets to calculate the absolute shortest clearance line.
- **Expert-Mapped Corridors:** Manual coordinate selection based on expert navigation rules for Marsyard slope avoidance.
- **Euclidean Baselines:** Straight-line vectors between start and goal used in obstacle-free tests (e.g., `empty_straight`).

Part 4: Mock Planner Algorithms

The package provides a mock planner (`mock_planner.py`) implementing the standard Nav2 Action interface. It provides a straight-line driver and an A* planner to verify benchmark scoring.

1. Straight-Line Mode (Lower-Quality Benchmark Target)

Directly calculates the vector between start and goal, adding waypoints every 10cm. It completely ignores obstacle data, simulating a failing or colliding planner.

2. A* Pathfinding Mode (High-Quality Benchmark Target)

The A* algorithm converts world coordinates to grid coordinates. It runs an 8-connected search (4 orthogonal moves at cost 1.0, and 4 diagonal moves at cost 1.414). Cells with a value ≥ 100 are treated as obstacles and skipped. For clear cells, the algorithm penalizes non-lethal costs using:

Move Cost = $\text{cost_multiplier} * (1.0 + \text{cell_cost} / 10.0)$

This forces A* to detour around rough gray regions rather than just running alongside obstacles. If A* fails (e.g. goal trapped), it falls back to a straight line path to return a result.

Part 5: The Benchmark Engine & Scorer

The engine manages the test sequence and evaluates the resulting paths against standard scoring equations. For each run, it queries a static path, computes safety metrics, injects dynamic obstacles into the occupancy grid, requests a replanned detour, and saves visual, JSON, and PDF reports.

1. The Importance of Dynamic Obstacles

Planetary navigation is highly unpredictable. Shifting slopes, debris, or other rovers can block path options. By injecting dynamic obstacles, the benchmarker measures the planner's **reactivity**, **replanning latency**, and **collision avoidance**. Planners that are slow to replan or fail to compute detours are penalized to 0.0 on the replanning score.

2. Automated Directory Cleanliness & Purging

To prevent stale or duplicate records from previous runs, the benchmarker runs automated purging:

- **Reports Cleanup:** Deletes previous files (`results.json`, `numerical_report.md`, `numerical_report.pdf`, and `result_scenario_*.png`) at the start of any test execution.
- **Synthetic Map Re-Generation:** Systematically deletes and recreates synthetic PNGs to reflect the current code configuration.

3. Single-Scenario Filtering & Report Scaling

When filtering execution to a single map using the `scenario_id` flag, the report generator dynamically hides skipped scenarios. The PDF summary table and detailed plots automatically resize to display only the metrics for the active run.

4. The Scoring Formulas & Passing Threshold

The total score is a weighted combination of six sub-scores out of 100. The **Pass Condition** is defined as a final **Path Planning Score $\geq 85.0/100$** :

Score Component	Weight	Mathematical Logic / Formula
-----------------	--------	------------------------------

Planning Success (S_success)	25%	100 if path returned successfully; 0 otherwise.
Planning Time (S_time)	15%	If $T \leq 2.0s$: 100. If $T \geq 10.0s$: 0. Else: $100 * (1 - (T - 2.0)/8.0)$
Obstacle Avoidance (S_obstacle)	25%	100 if robot footprint radius never intersects cell ≥ 100 ; else 0.
Path Cost (S_cost)	15%	$100 - C_{avg}$, where C_{avg} is the average cost of cells traversed.
Path Length (S_length)	10%	Ratio $R = \text{Planned}/\text{Reference}$. If $R \leq 1.0$: 100. If $R \geq 1.35$: 0. Else: $100 * (1 - (R-1.0)/0.35)$
Replanning (S_replan)	10%	100 if detour path computed in $\leq 2.0s$ with 0 collisions; else 0.

Part 6: Operations & Usage Guide

This section details the commands needed to compile the workspace, launch the nodes, verify ground-truth paths, and evaluate custom third-party path planning servers.

1. Configuration Modification (Maps, Starts, Goals)

To modify map inputs, change coordinates, or edit reference paths, edit the config file: config/scenarios.yaml. Below are the key parameter mappings:

YAML Key	Expected Format & Details	Purpose
map_image	"maps/filename.png"	Selects the structural environment layout.
start	[X_coord, Y_coord] (float)	Sets the starting location in meters.
goal	[X_coord, Y_coord] (float)	Sets the target location in meters.
robot_radius	float (e.g., 0.35)	Specifies the rover footprint boundary.
reference_path	List of coordinates [[x1,y1], [x2,y2]]	Defines the ground-truth optimal line.

2. Available Test Scenarios (Simple & Hard)

Nine scenarios are pre-configured to evaluate planner robustness:

- empty_straight: Straight-line path in open field (tests basic capability).
- scattered_rocks_detour: Obstacles in path, dynamic obstacle triggers detour (tests planning speed/replanning).
- canyon_gate_passage: Navigates a narrow gate corridor (tests narrow path alignment).
- marsyard_rough_slopes: ****[HARD]**** High-cost slope region in middle. Planner must detour around slopes to maintain low path cost.
- marsyard_labyrinth: ****[HARD]**** Winding maze walls. Tests footprint safety; planners without obstacle inflation will crash in tight corridors.
- canyon_gate_blocked: ****[HARD]**** Dynamic obstacle completely blocks the only gap. Tests planning failure and graceful abort handling.
- marsyard_snake_passage: ****[HARD]**** Winding narrow corridor S-curve (tests tight path smoothing, fine grid resolution, and footprint collision checking).
- dead_end_trap: ****[HARD]**** Dead-end U-shaped trap blocking the direct route. Tests heuristic trap escape and global detouring.
- crater_field: ****[HARD]**** Scattered slope crater field (variable cost gray levels). Tests cost-awareness and cost-weighted search optimization.

3. Build and Sourcing Setup

```
colcon build --packages-select global_path_benchmarking
source install/setup.bash
```

Output: Compiled C++ and Python binaries, environment setup scripts.

Saved Location: Binaries stored in build/, symlinks and setup paths created in install/.

4. Reference Path Footprint Verification

```
ros2 launch global_path_benchmarking benchmark.launch.py verify:=true
```

Output: Grayscale maps with plotted reference lines and transparent circular footprints.

Saved Location: Saved as reports/verify_scenario_<scenario_id>.png.

5. Running the Benchmark Suite (Standard A*)

```
ros2 launch global_path_benchmarking benchmark.launch.py
```

Output: Complete evaluation log, scoring breakdown, comparative plots, and Markdown summary.

Saved Locations:

- Raw JSON metrics: reports/results.json
- Printable Report: reports/numerical_report.md (contains PASS/FAIL status table)
- PDF Evaluation Report: reports/numerical_report.pdf (publication-ready compiled results)
- Result plots: reports/result_scenario_<scenario_id>.png (colored box showing PASS in green/FAIL in red).

6. Running with the Straight-Line Mock Planner

```
ros2 launch global_path_benchmarking benchmark.launch.py use_astar:=false
```

Output: Evaluation metrics showing high collision rates, high cost, and low scores due to obstacle intersections.

Saved Locations: Outputs saved to reports/results.json, reports/numerical_report.md, reports/numerical_report.pdf, and corresponding comparison PNGs.

7. Targeted Scenario Runs & System Cleanup

```
ros2 launch global_path_benchmarking benchmark.launch.py scenario_id:=canyon_gate_blocked clean:=true
```

Output: Spawns only the chosen scenario. Clears memory and kills orphan simulator/ROS nodes. Automatically purges all stale report artifacts from unrelated scenarios.

Saved Location: Outputs logged under reports/ (only for the selected ID).

8. Benchmarking a Custom Third-Party Path Planner

```
ros2 run global_path_benchmarking benchmarker --config config/scenarios.yaml
```

Output: Communicates with your path planning action server, scoring it against loaded configurations.

Saved Location: Results logged to reports/results.json, compiled into reports/numerical_report.md, and built as reports/numerical_report.pdf.